

PHast-R: Bit-Parallel Perfect Hashing with Rings of Patterns

Sebastiano Vigna

Dipartimento di Informatica, Università degli Studi di Milano

Abstract

PHast and its fast-to-build variant PHast⁺ are minimal perfect hash functions with the fastest known queries: a key is mapped to a bucket by a linear function of its hash, and a fixed-width per-bucket seed selects the positions of the keys of the bucket inside small overlapping slices of the output range. PHast⁺ uses additive placement, so that feasible seeds can be found with bit-parallel operations, but it needs more space than PHast, even in its variants with wrapping. We analyze where this space goes: with a minimal output range, every bumped key leaves a hole that must be encoded, and since all seeds move the keys of a bucket by the same amount, two keys that collide for a seed collide for (almost) all seeds: even with wrapping, such *self-collisions* account for almost 40% of the bumped keys. We then propose PHast-R, in which the lowest bits of the seed select one of R independent in-slice offsets (*patterns*), taken from bits that the computation of the bucket provides for free, and the seeds of a pattern move a key exactly once through a *ring* of equally spaced slots of its slice. Self-collisions disappear, and queries need just two shifts more than those of PHast⁺ with wrapping. Moreover, storing the set of used slots by residue classes turns a ring into a block of consecutive bits, so the seeds of a pattern that are feasible for a bucket are found by rotating and combining one block per key. With 8-bit seeds, PHast-R uses 1.93 bits per key against the 1.97 of PHast⁺ with wrapping, it is built in half the time, and its queries are faster; its space and its query time are close to those of PHast, which takes more than ten times as long to build. With 10-bit seeds it uses 1.85 bits per key against 1.87. Finally, we study the limits of the approach: we prove that when keys can move only within windows of W slots no placement of n keys on n slots, however found, can avoid bumping about $\sqrt{\pi n/2} - W$ keys, and we report evidence that the fraction of keys bumped by a sweep is bounded by the density that the sweep can reach, rather than by the flexibility of placement.

1 Introduction

A *minimal perfect hash function* (MPHF) maps a static set of n keys bijectively to $[0..n)$; it requires at least $\lg e \approx 1.443$ bits per key (we use $\lg$ for the binary logarithm and $\ln$ for the natural logarithm). Among the many practical constructions, *bucket placement* gives the fastest queries: keys are hashed to buckets, and a per-bucket seed chooses positions for the keys of the bucket so that no two keys collide.

PHast [1] pushes this approach to its limit. A key with hash c goes to bucket $\lfloor cB/2^{64} \rfloor$; the seed s of the bucket, stored in a plain array of S -bit values, places the key at $\lfloor c(m-L+1)/2^{64} \rfloor + p(s, c)$, that is, inside a *slice* of length L whose start grows linearly with c . Buckets for which no seed works are *bumped* (seed 0) and handled by further levels; an Elias-Fano sequence [3, 4] maps their outputs to the free slots of the first level. A query thus reads a single seed for almost all keys. Seeds are assigned by a sweep over the buckets that keeps a small window of buckets in a priority queue, favoring large buckets, and chooses for each bucket the feasible seed that minimizes the sum of the positions of its keys.

In PHast, $p(s, c)$ is a pseudorandom function, so finding the best seed means trying all $2^S - 1$ of them; construction takes hundreds of nanoseconds per key. PHast⁺ uses instead the additive placement $p(s, c) = (c \bmod L) + s - 1$: the feasibility of 64 consecutive seeds of a bucket is

the complement of the bitwise OR of 64-bit windows of the occupancy bitmap, one per key, and the best seed is the first feasible one. Construction becomes an order of magnitude faster, but with 8-bit seeds space grows from 1.92 to 2.12 bits per key. PHast⁺ *with wrapping* uses $p(s, c) = (c + \delta s) \bmod L$ for a small multiplier δ , so that keys do not leave their slice, and reduces space to 1.97 bits per key ($\delta = 3$) at the cost of a slower construction.

This paper makes four contributions:

- an analysis of the space of PHast⁺, with and without wrapping, showing that its excess comes from bumping, and that a large part of the bumping is caused by self-collisions inherent to additive placement (Section 2);
- PHast-R, a structure in which the lowest bits of the seed choose one of R independent patterns, obtained at no cost from the computation of the bucket, and the seeds of a pattern move a key through a ring of slots; rings make it possible to find all feasible seeds of a pattern by rotating blocks of bits (Section 3);
- an implementation in the Rust crate `sux` and an experimental comparison with the reference implementation of PHast and PHast⁺ (Section 4);
- a lower bound on the number of bumped keys that holds for every placement in which keys move within windows of fixed length, and experimental evidence that the fraction of bumped keys is limited by the density that the sweep can reach: once self-collisions are gone, more flexible placements or searches do not reduce it significantly (Section 5).

2 The Space of PHast⁺

All measurements in this section refer to 10^7 random keys and 8-bit seeds, with the space-optimal configurations of PHast⁺ (expected bucket size $\lambda = 5.25$ and slice length $L = 512$) and of PHast⁺ with wrapping ($\delta = 3$, $\lambda = 5$, $L = 1024$). They have been obtained on the reference implementation (the `ph` crate, using GxHash as hash function), whose first level we inspect through read-only accessors.

Holes are bumped keys. The first level of PHast⁺ has output range $m = n$. Every key is either placed in a distinct slot or bumped, so the number of free slots (*holes*) is exactly the number of bumped keys. Each hole costs about $2 + \lg(1/\beta)$ bits in the Elias–Fano remapping, where β is the fraction of bumped keys, and each bumped key must be placed again by the following levels. With S -bit seeds the space is thus approximately

$$\frac{S}{\lambda(1 - \beta)} + \beta \left(2 + \lg \frac{1}{\beta} \right),$$

and since the entropy of a random set of βn holes is already about $\beta n(\lg(1/\beta) + \lg e)$, the encoding of holes cannot be substantially improved: *the only lever is β* . Without wrapping $\beta = 7.6\%$, and bumping costs 0.59 of the 2.12 bits per key (0.47 for Elias–Fano, 0.13 for the further levels); with wrapping $\beta = 4.3\%$, and bumping costs 0.37 of the 1.97 bits per key (0.30 and 0.07). We will show in Section 5 that on large key sets no placement, however found, can avoid bumping altogether.

Additive placement is rigid. Additive placement translates a fixed pattern: two keys of a bucket whose base positions coincide collide for every seed. For a bucket of size k this happens with probability about $k(k - 1)/(2L)$, so without wrapping a fraction $\lambda^2/(2L) \approx 2.7\%$ of the buckets cannot be placed at all (we measure 2.6%); they are 38.5% of the bumped buckets and contain about $(\lambda^2 + 2\lambda)/(2L) \approx 3.7\%$ of the keys (we measure 3.6%). Wrapping does not change

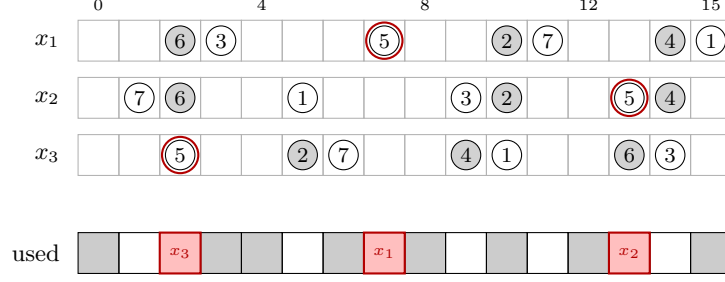


Figure 1: The slots of the three keys x_1 , x_2 , x_3 of a bucket for each seed, with 3-bit seeds, $R = 2$ patterns, and slices of length $L = 16$ (for simplicity, the three slices coincide). Even seeds (gray) select pattern 0, odd seeds (white) pattern 1, and seed s moves a key by $2s$ slots, modulo 16, from the offset of its pattern: the seeds of a pattern thus move a key by multiples of the stride $T = 4$ through a ring of four slots. In pattern 0 the keys x_1 and x_2 have the same offset, so they collide for all even seeds, as they would do for all seeds in PHast⁺; in pattern 1 they do not. The last row shows the slots that are already used: the only feasible seed is 5.

the picture: two keys with the same base position are moved together around their slices, and collide for all seeds but the few for which exactly one of them has wrapped. The slices of the variant with wrapping are longer, so these *self-collisions* are half as frequent, but so is bumping: they happen in 1.2% of the buckets, which contain 1.7% of the keys, that is, 39% of the bumped keys. Moreover, consecutive seeds are strongly correlated trials. Seeds, on the other hand, are almost incompressible: their empirical entropy is 7.71 bits without wrapping and 7.89 bits with wrapping.

3 PHast-R

Placement. Each key has a 64-bit hash h . Let B be the number of buckets, which we require to be odd, and denote with $\text{hi}(x)$ and $\text{lo}(x)$ the upper and lower 64 bits of a 128-bit product. The bucket of a key is $\text{hi}(hB)$, and the start of its slice is $\sigma(h) = \text{hi}(h(m - L + 1))$, as in PHast⁺ with wrapping. Let moreover $R = 2^\rho$ be the number of *patterns*, and $L = 2^\ell$ the slice length, with $R\ell \leq 64$ and, for the time being, $\ell \geq S$. We let $o = \text{lo}(hB)$: the r -th pattern assigns to the key the in-slice offset

$$\omega_r(o) = \lfloor o/2^{64r/R} \rfloor \bmod L,$$

that is, the ℓ bits of o starting at bit $64r/R$. A seed $s \in [1 \dots 2^S)$ selects the pattern $r = s \bmod R$, and the key is mapped to

$$\sigma(h) + (\omega_r(o) + s \cdot 2^{\ell-S}) \bmod L.$$

As in PHast, seed 0 means that the bucket is bumped.

The seeds of pattern r are $r, r + R, r + 2R, \dots$, so they move a key by multiples of the *stride* $T = RL/2^S$: as the seed varies, the key goes exactly once through the $2^S/R = L/T$ slots of its slice that are congruent to $\sigma(h) + \omega_r(o) + r2^{\ell-S}$ modulo T . We call these slots a *ring* (Figure 1). In other words, whereas in PHast⁺ all seeds move the same rigid pattern of base positions, in PHast-R the seed first chooses among R patterns, and then moves the keys along the rings of the chosen one. Two keys of a bucket collide for all seeds of a pattern if they are mapped to the same slot by the first one, but since the offsets of different patterns are disjoint blocks of bits of o , this happens independently in each pattern: a bucket of size k has no usable pattern with probability about $(k(k-1)/2L)^R$, so self-collisions disappear already for $R = 2$. Moreover, different patterns provide (almost) independent trials.

The value o costs nothing, as it is the other half of the product that yields the bucket, and it is a good source of offsets: for the keys of bucket b we have $hB = b \cdot 2^{64} + o$, so o is the position

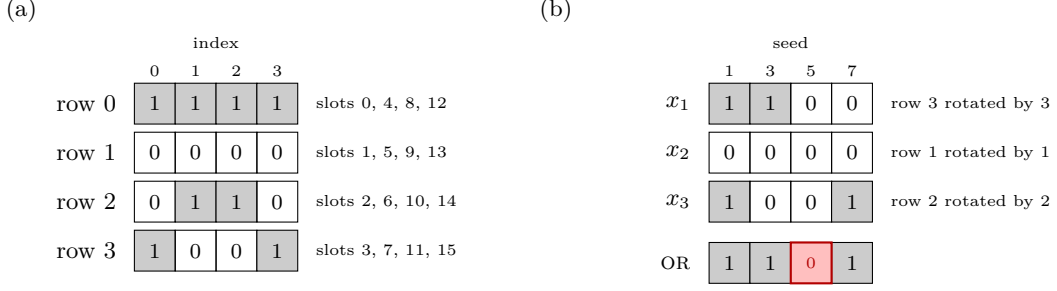


Figure 2: Searching for the seeds of pattern 1 that are feasible for the bucket of Figure 1. (a) The set of used slots is stored by residue classes modulo the stride $T = 4$: bit q of row ϱ is associated with slot $qT + \varrho$. (b) The ring of a key is a row (in general, a block of consecutive bits of a row), and the seeds of the pattern go through its bits cyclically, starting from the index of the slot of the first seed: for example, seed 1 maps x_1 to slot 15, which has index 3 in row 3. The bits of the rings, rotated by these indices, are combined by a bitwise OR: the zeros of the result are the feasible seeds.

of h in the interval of hashes of the bucket, scaled to 64 bits, and it is uniformly distributed if h is. (Since B is odd, also the lowest bits of o depend bijectively on those of h .) In particular, the offsets of the keys of a bucket are independent of each other for every pattern, which would not be true of blocks of bits of h , as the keys of a bucket share the upper $\approx \lg B$ bits of h .

Queries. A query computes hB , reads the seed s of bucket $\text{hi}(hB)$, and, unless $s = 0$, computes the formula above. There is no need to extract r : since $(s \cdot 64/R) \bmod 64 = r \cdot 64/R$, the offset $\omega_r(o)$ is obtained by shifting o right by $s \cdot 64/R$ modulo 64, and the lower bits of the result are extracted by the reduction modulo L that follows the addition of $s \cdot 2^{\ell-S}$. After reading the seed, PHast⁺ with wrapping multiplies it by δ , adds the hash, reduces the result modulo L , and adds $\sigma(h)$; PHast-R performs the same operations, with a shift in place of the multiplication and o in place of the hash, after computing $s \cdot 64/R$ and shifting o : two shifts more. With $R = 1$ and $\delta = 2^{\ell-S}$, PHast-R is essentially PHast⁺ with wrapping (which, however, uses odd multipliers).

Search. In PHast⁺ the positions of a key for consecutive seeds are consecutive slots, so the seeds that are feasible for a bucket are found by combining windows of the bitmap of used slots; with wrapping, however, the range of seeds must be split at the points at which some key wraps, and with $\delta > 1$ only one bit out of δ of each window is meaningful. In PHast-R the positions of a key for the seeds of a pattern are T slots apart, but since T is a power of two and it divides L we can make them adjacent again, and get rid of wrapping at the same time: we store the set of used slots by residue classes modulo T , that is, as T rows in which bit q of row ϱ is associated with slot $qT + \varrho$. The ring of a key is then a block of L/T consecutive bits of a row, and the seed of index j of a pattern (i.e., $jR + r$) maps the key to the bit of its ring of index $(x + j) \bmod (L/T)$, where x is the index for the first seed. Thus, the bits telling which seeds of a pattern map a key to a used slot are the bits of its ring rotated by x , and combining them for all the keys of the bucket by a bitwise OR we obtain a block whose zeros are the feasible seeds (Figure 2). For $S = 8$ and $R = 4$ a ring has 64 slots, whatever the slice length: each pattern requires just reading and rotating a word for each key. Note that in the first pattern the first seed is 0, which is not available.

Among feasible seeds we choose, as in PHast, the one minimizing the sum of the slots of the keys. For a bucket of size k , the sum for the seed of index j of a pattern is the sum for the first seed of the pattern plus kjT , minus L for each key that has gone past the end of its ring, that is, for each key such that $x + j \geq L/T$. To count such keys we pack the indices x of the keys of the bucket into fields of a word that are large enough to contain the sum of two indices (for

$S = 8$ and $R = 4$, eight fields of eight bits): after adding j to all fields with a single addition, the bit of weight L/T of each field tells whether the corresponding key has gone past the end of its ring, and counting such bits gives the number of keys. Evaluating a seed thus requires a number of operations that does not depend on the size of the bucket (for buckets with at most eight keys, in the case above), and since buckets have usually just a few feasible seeds (Section 5) we evaluate all of them, with a single loop over the zeros of the blocks of all patterns that contains no other branch. When there are many feasible seeds (e.g., at the beginning of a sweep) we use instead the fact that in a pattern the sum increases with the index of the seed, except at the indices at which some key goes back to the first slot of its ring: thus, we need to consider only the first feasible index and, for each key, the first feasible index after the key went back, that is, at most $k + 1$ candidates per pattern. Finally, when marking the slots of the best candidate as used we check that no two keys of the bucket are mapped to the same slot, and otherwise we move to the next candidate.

Grouping and scheduling. The sweep needs the hashes of the keys of each bucket, but in no particular order, and we do not want to keep more than one copy of them. Thus, instead of sorting the hashes, as the reference implementation of PHast does, we distribute them into at most 256 *parts* made of a power of two of consecutive buckets: each thread computes the hashes of a chunk of the keys, and writes each of them at the end of the *region* of its part, in the order of the keys. Regions are sized before computing the hashes, assuming that hashes are uniform, with a slack of six standard deviations: the few hashes that do not fit their region, if any, are kept separately. A part is distributed into buckets only when a sweep needs it, into a buffer of the sweep (the part is distributed first into smaller parts that fit the cache, and then each smaller part into its buckets), so regions are never modified: after the sweep, the keys of bumped buckets, which must be passed to the next level, are found by walking the keys of each chunk in order with a byte per key recording its part, and reading their hashes from the regions. Writing to at most 256 regions at a time and working on parts of about 150 KB is efficient on any current architecture, and the construction needs, besides the hashes, just a byte per key: about 11 bytes per key overall, against the 10 of the reference implementation, which sorts the hashes in place, and hashes all keys again to find those of bumped buckets.

The priority of a bucket depends on its size and decreases with its index, and buckets enter the window of the sweep in order of index: thus, the buckets of each size in the window form a queue, and the bucket with the highest priority is the first bucket of one of the queues. Instead of a priority queue we keep a queue for each size: extracting the next bucket requires scanning the first elements of a few queues, and adding a bucket takes constant time. Finally, the set of used slots of the level, which is necessary to build the following levels, is a byproduct of the sweep: the bits of the slots that are no longer reachable are copied to the bitmap of the level when their part of the set of used slots is recycled.

Slice length. The slice length of a level with output range m is the smallest power of two larger than $m/2$, up to a maximum, which is a parameter: we use $L = 1024$ for $S = 8$ and $L = 2048$ for $S = 10$, as PHast⁺ with wrapping does. The slices of small levels can thus be shorter than the number of seeds, that is, $\ell < S$: in this case seed s moves a key by s , so the stride is R , rings have L/R slots (a single slot if $L < R$), and different seeds of a pattern map the keys of a bucket to the same slots. The priority of a bucket has the same form as in PHast (a weight depending on the size of the bucket, extrapolated linearly beyond size 7, minus 1024 times the index of the bucket), and we use the weights of PHast⁺ with wrapping.

Levels. Levels after the first use the same scheme with range equal to their number of keys, and an Elias–Fano sequence maps their outputs to the holes of the first level. In PHast⁺ the keys of bumped buckets are hashed again at each following level with a different seed, and the

new hash determines their bucket, their slice, and their offsets. In PHast-R a bumped key is hashed again just once, with a different seed: if h' is this second hash, the value playing the role of h at level $i > 1$ is

$$\text{hi}((h \oplus z_i)(h' \oplus c)) \oplus \text{lo}((h \oplus z_i)(h' \oplus c)),$$

where z_i is a *salt* associated with the level and c is a constant. Thus, after a bumped bucket is detected and the second hash is computed the key is no longer necessary: the levels after the first are accessed by code that depends just on two integers, which can be kept separate from the code accessing the first level without the need of preserving the key.

Colliding signatures. Keys with the same 64-bit hash at the first level have the same slice and the same offsets in all patterns, so they collide for every seed and their bucket is bumped; at the following levels their values depend also on the second hash, and thus they are separated. As in PHast⁺, 64-bit hashes are therefore sufficient for any number of keys, and no larger signature is necessary. Construction would fail only if two keys had the same pair of hashes, which happens in practice only for duplicate keys. Since duplicate keys are always bumped from the first level, to report them we examine the buckets of the following levels: keys of a bucket with the same value are hashed with a third seed, and keys whose hashes coincide again are considered duplicates.

Last level. When at most 4096 keys are left, a last level uses the same placement scheme with a range larger by 25% plus 16 slots, expected bucket size $\min(\lambda, 3)$, and no bumping, changing the salt (and, every eight attempts, enlarging the range) in the unlikely case of failure. PHast⁺ behaves differently: when at most 8192 keys are left (4096 for the variants with wrapping), it builds a level of regular PHast (pseudorandom placement, all 256 seeds of 8 bits tried for each bucket, expected bucket size 4, no bumping) with range $1.2(k + 10)$, where k is the number of keys, rehashing all keys with a new seed in case of failure.

Parallel construction. As in PHast, buckets are split into chunks separated by gaps of

$$\left\lfloor \frac{LB}{m - L + 1} \right\rfloor + 1$$

buckets: the slice of bucket b starts at about $b(m - L + 1)/B$, and the positions of a key lie at most $L - 1$ slots after the start of its slice, so (considering the rounding of slice starts) the positions of different chunks cannot overlap. Chunks are swept in parallel. Gaps are then processed differently: PHast processes them serially, continuing for each gap the sweep of the previous chunk (whose state, including the priority queue, is preserved) after adding to its occupancy the positions of the first buckets of the following chunk; PHast-R sweeps all gaps in parallel, each with a new sweep in which the positions of the buckets of the neighboring chunks (on both sides) are marked as used.

Summary of the differences with PHast⁺. With respect to PHast⁺ with wrapping, in PHast-R the seed selects a pattern, besides moving the keys, and the offsets of the patterns are taken from the lower half of the product yielding the bucket, rather than from the hash; the multiplier of the seed is the power of two $L/2^S$, so that the seeds of a pattern form rings, rather than a small odd number; the set of used slots is stored by residue classes, feasible seeds are found by rotations, and their sums of slots are computed on packed indices (Search); hashes are distributed into parts, rather than sorted, parts are distributed into buckets only when a sweep needs them, the keys of bumped buckets are found by walking the keys, the priority queue of the sweep is replaced by a queue for each bucket size, and the set of used slots of a level is a byproduct of the sweep (Grouping and scheduling); the keys of bumped buckets are hashed just once more, rather than at each level (Levels); the last level uses the same scheme as the

Table 1: Comparison on 10^7 keys (single-threaded construction).

Method	Configuration	bits/key	build [ns/key]	query [ns]
PHast ⁺	$S = 8, \lambda = 5.25$	2.116	68	23.8
PHast ⁺ wrap $\delta = 1$	$S = 8, \lambda = 5.25$	2.096	80	22.7
PHast ⁺ wrap $\delta = 2$	$S = 8, \lambda = 5$	2.013	97	21.1
PHast ⁺ wrap $\delta = 3$	$S = 8, \lambda = 5$	1.970	118	22.1
PHast	$S = 8, \lambda = 4.5$	1.922	810	20.9
PHast-R	$S = 8, L = 1024, \lambda = 4.5$	1.929	56	20.5
PHast-R	$S = 8, L = 1024, \lambda = 4.75$	1.921	54	21.1
PHast-R	$S = 8, L = 1024, \lambda = 5$	1.925	54	21.7
PHast ⁺	$S = 10, \lambda = 5.15$	2.085	81	25.3
PHast ⁺ wrap $\delta = 1$	$S = 10, \lambda = 6.2$	1.905	111	24.5
PHast ⁺ wrap $\delta = 2$	$S = 10, \lambda = 5.9$	1.883	172	23.6
PHast ⁺ wrap $\delta = 3$	$S = 10, \lambda = 6$	1.872	242	25.3
PHast	$S = 10, \lambda = 6.05$	1.854	2212	25.4
PHast-R	$S = 10, L = 2048, \lambda = 6$	1.852	118	24.8
PHast-R	$S = 10, L = 2048, \lambda = 6.25$	1.857	120	25.2

other levels instead of regular PHast (Last level); and gaps between parallel chunks are swept in parallel by independent sweeps instead of serially by continuing the previous sweep (Parallel construction). Slice lengths and bucket priorities are the same; the suggested expected bucket size for 8-bit seeds is 4.5 instead of 5.

4 Experiments

We ran all experiments on an Intel Xeon E-2388G (8 cores, 3.2 GHz) with 128 GiB of RAM, compiling with Rust 1.98 at optimization level 3 with thin link-time optimization. Keys are distinct 64-bit integers. The reference implementation is the `ph` crate by Beling (`Function2` with the `ShiftOnly`, `ShiftOnlyWrapped< $\delta$ >` and `SeedOnly` seed choosers for PHast⁺, PHast⁺ with wrapping and PHast, respectively). To make query times comparable we fed both implementations the same hash function, GxHash with a seed, as in the experiments of the PHast paper. Construction times are single-threaded unless otherwise noted and include hashing; query times are medians over nine rounds of $2 \cdot 10^6$ queries of random keys of the set, and the rounds of the different structures are interleaved. PHast-R uses $R = 4$ patterns, with byte seeds for $S = 8$ and bit-packed seeds read without alignment for $S = 10$.

Table 1 and Figure 3 show the results on 10^7 keys. With 8-bit seeds and the suggested expected bucket size $\lambda = 4.5$, PHast-R uses 0.04 bits per key less than the best variant of PHast⁺ with wrapping, and it is built in less than half the time (it is faster to build even than PHast⁺ without wrapping). Its queries are faster than those of all variants of PHast⁺ (6% faster than those of the variant using the least space), and slightly faster than those of PHast, which uses 0.4% less space but takes fifteen times as long to build. The speed of queries is due to bumping: PHast-R bumps just 1.5% of the keys from the first level, against the 4.3% of PHast⁺ with wrapping and $\delta = 3$, and queries for bumped keys are much slower than the others. The expected bucket size thus trades space for query time: with $\lambda = 4.75$ PHast-R bumps 2.5% of the keys, and it has the space and essentially the query time of PHast; with $\lambda = 5$ it bumps 3.7% of the keys, and its queries are still faster than those of PHast⁺ with wrapping and $\delta = 3$. With 10-bit seeds PHast-R uses less space than all variants of PHast⁺, and even slightly less than PHast; it is built in less than half the time of PHast⁺ with wrapping and $\delta = 3$, and its queries are slightly faster than those of the latter and of PHast (the variants of PHast⁺ with a smaller multiplier have faster queries, but they use more space).

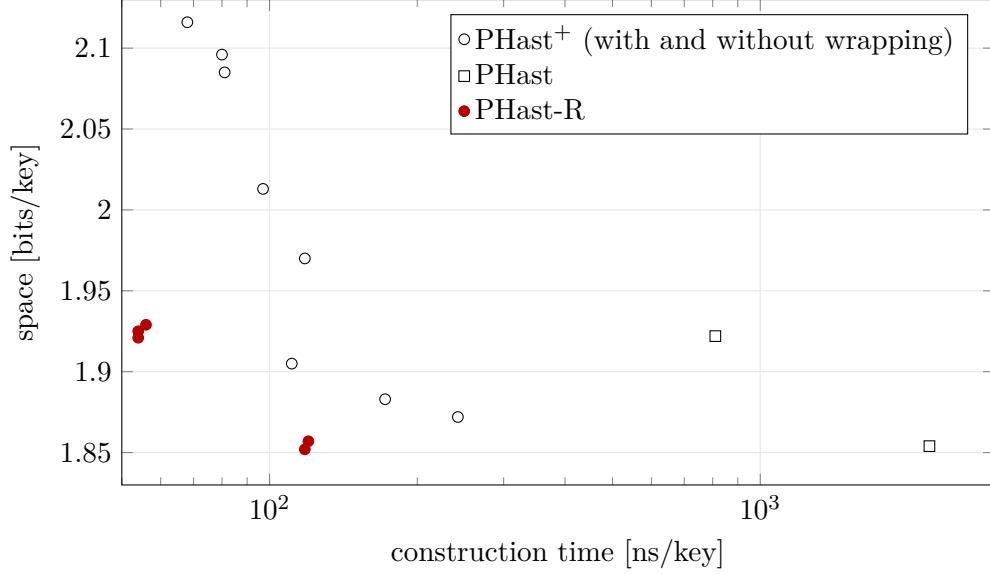


Figure 3: Space versus single-threaded construction time on 10^7 keys (data of Table 1).

Table 2: Comparison on 10^8 keys, with single-threaded and 8-threaded construction.

Method	Configuration	bits/key	build [ns/key]		query [ns]
			1 thread	8 threads	
PHast ⁺	$S = 8, \lambda = 5.25$	2.115	68	15.6	35.9
PHast ⁺ wrap $\delta = 3$	$S = 8, \lambda = 5$	1.968	129	23.1	33.1
PHast	$S = 8, \lambda = 4.5$	1.921	817	–	30.5
PHast-R	$S = 8, L = 1024, \lambda = 4.5$	1.927	59	9.4	30.0
PHast-R	$S = 8, L = 1024, \lambda = 4.75$	1.919	56	9.7	31.1
PHast-R	$S = 8, L = 1024, \lambda = 5$	1.924	57	9.9	32.4
PHast ⁺ wrap $\delta = 3$	$S = 10, \lambda = 6$	1.869	254	41.2	35.8
PHast-R	$S = 10, L = 2048, \lambda = 6$	1.849	121	17.3	35.3

Table 2 shows that the picture does not change on 10^8 keys, where queries are dominated by cache misses: with 8-bit seeds and $\lambda = 4.5$ PHast-R is built in less than half the time of PHast⁺ with wrapping, and its queries are 9% faster; they are also slightly faster than those of PHast. With 8 threads PHast-R builds in 41% of the time of PHast⁺ with wrapping with 8-bit seeds, and in 42% of the time with 10-bit seeds.

4.1 Large Key Sets

Table 3 reports the results on 10^9 keys. Space does not depend on the number of keys, and the relative performance of the structures is the same as on smaller key sets: with 8-bit seeds PHast-R is built in half the time of PHast⁺ with wrapping with one thread, and in 41% of the time with eight, its queries are 9% faster, and they are as fast as those of PHast. At this scale, however, some phenomena become more visible, and we used them to tune the structure.

Bumped keys. Table 4 shows the time of queries for the keys of the first level only, measured on the same structures. It is essentially the same for all structures, as it is dominated by the access to the seed (and, in our benchmark, to the key), and it grows with the size of the array of seeds: the differences between the structures are due to bumped keys. In a workload in which all

Table 3: Comparison on 10^9 keys, with single-threaded and 8-threaded construction.

Method	Configuration	bits/key	build [ns/key]		query [ns]
			1 thread	8 threads	
PHast ⁺	$S = 8, \lambda = 5.25$	2.115	74	17.5	57.1
PHast ⁺ wrap $\delta = 3$	$S = 8, \lambda = 5$	1.968	130	24.9	52.1
PHast	$S = 8, \lambda = 4.5$	1.920	818	–	47.6
PHast-R	$S = 8, L = 1024, \lambda = 4.5$	1.927	66	10.3	47.3
PHast-R	$S = 8, L = 1024, \lambda = 4.75$	1.919	64	10.8	48.9
PHast-R	$S = 8, L = 1024, \lambda = 5$	1.924	63	10.8	51.2
PHast ⁺ wrap $\delta = 3$	$S = 10, \lambda = 6$	1.869	252	43.0	54.2
PHast-R	$S = 10, L = 2048, \lambda = 6$	1.848	131	19.0	54.2

Table 4: Query time in nanoseconds for the keys of the first level (“first”) and for all keys, and for all keys using huge pages; the second column is the percentage of keys bumped from the first level.

Structure	bumped	10 ⁷ keys		10 ⁸ keys		10 ⁹ keys		huge pages	
		first	all	first	all	first	all	10 ⁸	10 ⁹
PHast ⁺ wrap $\delta = 3, \lambda = 5$	4.3	19.5	22.0	28.1	32.9	44.1	52.4	26.9	32.1
PHast, $\lambda = 4.5$	1.4	20.1	20.9	28.9	30.5	45.1	48.1	25.2	28.9
PHast-R, $\lambda = 4.5$	1.5	19.6	20.5	28.1	29.9	44.7	47.4	24.6	28.3
PHast-R, $\lambda = 4.75$	2.5	19.5	21.0	28.0	30.8	44.5	49.4	25.4	29.6

keys are queried, a query for a bumped key takes about 60 ns more than a query for a key of the first level on 10^7 keys, 115 ns more on 10^8 keys, and about 200 ns more on 10^9 keys: besides accessing another level and the Elias–Fano sequence, a bumped key causes the misprediction of the test on the seed, which interrupts the overlapped execution of the following queries. Thus, each percent of bumped keys costs about 0.6 ns on 10^7 keys, but 2 ns on 10^9 keys: the larger the key set, the more a small expected bucket size is convenient. This is the reason why we suggest $\lambda = 4.5$ instead of $\lambda = 4.75$, which uses slightly less space.

Page size. With 10^9 keys the seeds of the first level occupy 222 MB, and with pages of 4 KiB almost every access to a seed causes a TLB miss, besides a cache miss. The last two columns of Table 4 show the query time when memory is allocated using transparent huge pages (on our system they must be requested, which we did through the tunable `glibc.malloc.hugetlb` of the GNU C library): queries are 18% faster on 10^8 keys, and 40% faster on 10^9 keys, for all structures. For very large key sets the page size is thus more important than any other parameter.

Construction. The construction time per key of PHast-R grows slowly with the number of keys (55, 59 and 66 ns with one thread on 10^7 , 10^8 and 10^9 keys, respectively), as parts are distributed into smaller parts that fit the cache whatever their size (Section 3); the growth is due to the walk finding bumped keys, which accesses randomly a bit vector with a bit per bucket. Table 5 shows how construction scales with the number of threads: the speedup is limited by the phases that are bound by memory bandwidth and latency, that is, hashing, the distribution into parts, and the walk, which with eight threads take about half of the time. The peak memory usage is close to that of the reference implementation of PHast⁺, which sorts hashes in place and hashes all keys again to find those of bumped buckets. Huge pages are useful for construction, too, as page faults on the large arrays of hashes are a significant part of its cost: with them, PHast-R builds 10^9 keys in 61 ns per key instead of 66 with one thread, and in 9.7 instead of

Table 5: Construction on 10^9 keys ($S = 8$): time in nanoseconds per key by number of threads (the last column uses both hardware threads of each core), speedup with eight threads, and peak memory usage in bytes per key, excluding the keys.

Structure	threads					speedup	bytes/key
	1	2	4	8	16		
PHast-R, $\lambda = 4.5$	64.4	34.2	18.4	10.5	9.6	6.1	10.7
PHast ⁺ wrap $\delta = 3$, $\lambda = 5$	129.8	70.3	39.1	24.9	20.9	5.2	9.9

10.3 with eight.

5 The Limits of the Approach

The first level of PHast-R still bumps a few percent of the keys, and bumping is where space goes: it is natural to ask whether a more flexible placement, or a more powerful search, could get rid of it. In this section we show that the answer is negative, for two different reasons. The first reason is structural, and it applies to every placement scheme of the PHast family: no placement of the keys, however found, can avoid bumping. The second reason is experimental: the fraction of bumped keys settles on a value that depends on the density that the sweep can reach, and attempts to reduce it by making placement more flexible, or the search more powerful, have little effect.

5.1 A Lower Bound on Bumping

In all structures of the PHast family a key whose slice starts at σ can be placed only in the *window* $[\sigma \dots \sigma + W)$, where $W = L$ for PHast, for PHast⁺ with wrapping, and for PHast-R, and $W = L + 2^S - 2$ for PHast⁺ without wrapping. The seed of a bucket chooses positions within the windows of its keys, but the windows themselves depend only on the hashes of the keys. This simple constraint is sufficient to force bumping.

Proposition 1. *Consider n keys and n slots, and assume that, for some $W \geq 1$, each key x can be placed only in the slots $[\sigma_x \dots \sigma_x + W)$. Let*

$$X(t) = t - |\{x \mid \sigma_x < t\}| \quad (0 \leq t \leq n),$$

and let $R = \max_t X(t) - \min_t X(t)$ be the range of X . Then, every injective placement of a subset of the keys leaves at least $R - W + 1$ keys unplaced.

Proof. For $0 \leq s \leq t \leq n$ the number of keys x with $s \leq \sigma_x < t$ is $t - s - (X(t) - X(s))$. Only these keys can occupy the $t - s - W + 1$ slots in $[s + W - 1 \dots t)$, so at least $X(t) - X(s) - W + 1$ of these slots are empty. Symmetrically, these keys can occupy only the at most $t - s + W - 1$ slots in $[s \dots t + W - 1)$, so at least $X(s) - X(t) - W + 1$ of them are unplaced. Since there are as many keys as slots, the number of unplaced keys is equal to the number of empty slots, and it is thus at least $|X(t) - X(s)| - W + 1$ for all $s \leq t$. Choosing as s and t the points at which X attains its minimum and its maximum, in the appropriate order, completes the proof. $\square$

The proposition is a quantitative version of the (easy) necessity of Hall’s condition [6] for the bipartite graph connecting keys and slots. In the PHast family slice starts are computed from a hash by inversion in fixed-point arithmetic, so we can model them as independent and uniformly distributed. If the σ_x are independent and uniform in $[0 \dots n)$, then $R = nV_n$, where V_n is Kuiper’s statistic [9] of the values σ_x/n , that is, the sum of the maximum positive and negative deviations of their empirical distribution from the uniform distribution (the compression of slice starts into

Table 6: Fraction of keys bumped from the first level, and space, for variants of the construction (10^7 keys, $S = 8$, $L = 1024$, $\lambda = 5$). These values do not depend on the hardware.

Construction	bumped	bits/key
Wrapping ($\delta = 3$, as in PHast ⁺)	4.35%	1.970
Wrapping, repair of depth 1	3.66%	1.920
Wrapping, repair of depth 2	3.55%	1.913
Rings of patterns ($R = 4$, PHast-R)	3.73%	1.925
Rings of patterns, repair of depth 1	3.61%	1.916

$[0 \dots n - W + 1)$ changes R by at most $W - 1$). By Donsker’s theorem [2] and the continuity of the range, $R/\sqrt{n}$ converges in distribution to the range of a standard Brownian bridge, which by Vervaat’s transformation [12] has the same distribution as the maximum of a standard Brownian excursion, whose mean is $\sqrt{\pi/2}$ [8]. Since the Dvoretzky–Kiefer–Wolfowitz inequality [11] gives $R/\sqrt{n}$ exponentially decreasing tails, the convergence extends to expectations, and we conclude that

$$\mathbf{E}[R] = \sqrt{\frac{\pi n}{2}}(1 + o(1)).$$

We verified the estimate experimentally: over 200 random sets of 2^{12} , 2^{16} , and 2^{20} keys, the average range is within 1% of $\sqrt{\pi n/2}$.

The bound is irrelevant in terms of space. For PHast-R with $S = 8$ and $L = 1024$ we have $W = 1024$, so on 10^7 keys at least about 2900 keys (0.03%) must be bumped, and on 10^9 keys about 38 600 (0.004%), whereas, as we will see shortly, the first level of PHast-R bumps a few percent of the keys. Its importance lies in the fact that it applies to *every* placement: as soon as $\sqrt{\pi n/2}$ is significantly larger than W , that is, beyond a few hundred thousand keys, no amount of search can make a first level with $m = n$ free of bumping. In particular, it rules out the possibility of obtaining a bump-free version of PHast-R, whose space would be just S/λ bits per key, by a backtracking search in the style of CONSENSUS [10], which combines the search and the encoding of seeds to get arbitrarily close to the lower bound for MPHFs. For such a search to succeed, keys should be first split into parts of exactly known size C , with $\sqrt{\pi C/2}$ significantly smaller than W , as it happens (albeit for different reasons) in Bucketed CONSENSUS-RecSplit, whose first step is a minimal k -perfect hash function.

Even within such parts, however, a second obstacle appears. Under the minimum-sum rule, whether a slot will be occupied is decided by buckets whose slices start up to W slots before it, that is, about W/λ buckets before the slot becomes unreachable, so a depth-first search that detects a failure when a slot becomes unreachable must exhaust the alternatives of all such buckets before it can change the culprit. In a prototype using chained seeds as in CONSENSUS (the offsets of a bucket depend on the seeds of the previous buckets, so that backtracking rerandomizes the following ones), a cyclic output range to avoid boundary effects, and a check that all unreachable slots are occupied, the search never placed more than 160 buckets, even with more than two bits of slack per bucket and a budget of thousands of placements per bucket.

5.2 The Density Limit

The bump rate of PHast-R is two orders of magnitude larger than the lower bound of Proposition 1, so the latter is not what limits it in practice. On the other hand, self-collisions involve 1.7% of the keys (Section 2), but getting rid of them reduces the fraction of bumped keys much less: with $\lambda = 5$, from 4.35% to 3.73%. To understand why, we experimented with several variants of the construction.¹

¹The variants discussed in this section are not part of the final implementation, but they are available in the history of its repository.

The first variant is a cuckoo-style *repair*, similar to the eviction mechanism of PtrHash [5]: when no seed is feasible for a bucket, we look for the seeds for which exactly one of its keys is mapped to a used slot (they can be found in a bit-parallel fashion, too), evict the bucket owning the slot, place the failing bucket, and place the evicted bucket again, possibly evicting in turn another bucket up to a maximum depth; if the evicted bucket cannot be placed, the eviction is undone, and another seed is tried. Table 6 shows the results. Repair cannot do anything for self-collisions, yet when applied to placement with wrapping it reaches (at a significant cost in construction time) the bump rate of PHast-R, and repair of depth two improves on it only slightly; applied to PHast-R, it is almost useless. All variants converge to a bump rate of about 3.6%.

The reason is a feedback effect. Since with $m = n$ holes and bumped keys are the same thing (Section 2), every additional bucket that is placed fills slots that would otherwise have remained empty: the region around the frontier of the sweep becomes denser, and more buckets fail. The sweep settles into an equilibrium that depends on the density at which buckets no longer fit, rather than on the placement scheme: in this equilibrium, buckets containing a single key are always placed, and buckets containing two keys fail one or two times out of a hundred, so holes would be easily filled by small buckets; but with $\lambda = 5$ only 0.7% of the keys are in buckets of size one, and 3.4% in buckets of size two. When it is placed, a bucket of PHast-R has on average 4.5 feasible seeds, almost independently of its size (which is the purpose of priorities); the number of feasible seeds, however, has a large variance, and 3–4% of the buckets have none. The two bits or so of each seed that pay for this slack are the price of a greedy choice with seeds of fixed width, and they account for most of the distance of PHast-R from the lower bound.

Other attempts to move the equilibrium confirm this picture (the following figures refer to $3 \cdot 10^6$ keys and $\lambda = 5$, where PHast-R bumps 3.7% of the keys):

- *Overloading.* A hole costs more than three times as much as a bumped key, so we can give the first level fewer slots than keys, so that fewer holes are left, let each following level use directly a part of the remaining output range, and remap only the last keys to the holes of all levels. However, each percent of overload removes just 0.19–0.28% of holes, and the trade is convenient only above 0.25%.
- *Skewed bucket sizes.* With seeds of fixed width, a suitable nonuniform distribution of bucket sizes is much better than a uniform one when buckets are placed globally by decreasing size [7]. In a sweep, sizes must be mixed locally: we tried periodic sequences of expected sizes, but bumping increases with the skew (from 3.8% to 6.3% in our experiments), as large buckets do not find the almost empty regions they need, even when they are processed first.
- *A different choice among feasible seeds.* Choosing the seed minimizing the sum of a concave function of the positions of the keys, which favors seeds filling old holes, helps only slightly (logarithms give 3.65%); convex functions are harmful.
- *Unbounded repair.* Letting repairs proceed with no bounds, as in PtrHash, makes things worse: chains of evictions drift backwards, as the owner of a blocking slot is usually a bucket whose slice starts earlier than that of the evicting one, and when a chain reaches a bucket whose window is entirely occupied every eviction causes several further evictions.

In practice, then, once self-collisions are out of the way the bump rate is determined by the expected bucket size and by the number of seeds, and the only effective levers are those of PHast: smaller buckets, which cost S/λ bits per key, or larger seeds.

6 Conclusions

The excess space of PHast⁺ is the cost of bumping, and a large part of the bumping is caused by the rigidity of additive placement, even with wrapping. Selecting one of a few independent patterns with the lowest bits of the seed removes the rigidity at the cost of two shifts per query, since the offsets of the patterns come for free with the computation of the bucket; and moving the keys along rings preserves bit-parallelism, as feasible seeds are found by rotating blocks of bits of a set of used slots stored by residue classes. The resulting structure, PHast-R, uses less space than PHast⁺ with wrapping, is built in half the time, and has faster queries. Bumping has, however, intrinsic limits: no placement in which keys move within windows of W slots can avoid bumping about $\sqrt{\pi n/2} - W$ keys, and in practice the fraction of bumped keys is determined by the density that the sweep can reach, which more flexible placements and searches do not change.

There are several open problems and further research directions. On the practical side, the bucket priorities and the geometry of small levels should be tuned for rings, access to bit-packed seeds should be made faster, and the last levels could be replaced by a more compact structure. More interestingly, the lower bound of Section 5 does not apply to keys split into parts of exactly known size: whether bump-free placements, which would use just S/λ bits per key, can be found efficiently within such parts, in spite of the long-range constraints that defeat backtracking, is an open problem.

The implementation is available in the `sux` crate as `sux::func::PHastR`.

References

- [1] Piotr Beling and Peter Sanders. PHast — perfect hashing made fast. *CoRR*, abs/2504.17918, 2025.
- [2] Patrick Billingsley. *Convergence of Probability Measures*. Wiley, second edition, 1999.
- [3] Peter Elias. Efficient storage and retrieval by content and address of static files. *Journal of the ACM*, 21(2):246–260, 1974.
- [4] Robert M. Fano. On the number of bits required to implement an associative memory. Memorandum 61, Computer Structures Group, Project MAC, MIT, Cambridge, Mass., n.d., 1971.
- [5] Ragnar Groot Koerkamp. PtrHash: Minimal perfect hashing at RAM throughput. In *23rd International Symposium on Experimental Algorithms (SEA 2025)*, volume 338 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 21:1–21:21. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2025.
- [6] P. Hall. On representatives of subsets. *Journal of the London Mathematical Society*, 10(1):26–30, 1935.
- [7] Stefan Hermann, Hans-Peter Lehmann, Giulio Ermanno Pibiri, Peter Sanders, and Stefan Walzer. PHOBIC: Perfect hashing with optimized bucket sizes and interleaved coding. In *32nd Annual European Symposium on Algorithms (ESA 2024)*, volume 308 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 69:1–69:17. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024.
- [8] D. P. Kennedy. The distribution of the maximum Brownian excursion. *Journal of Applied Probability*, 13(2):371–376, 1976.
- [9] N. H. Kuiper. Tests concerning random points on a circle. *Proceedings of the Koninklijke Nederlandse Akademie van Wetenschappen, Series A*, 63:38–47, 1960.

- [10] Hans-Peter Lehmann, Peter Sanders, Stefan Walzer, and Jonatan Ziegler. Combined search and encoding for seeds, with an application to minimal perfect hashing. In *33rd Annual European Symposium on Algorithms (ESA 2025)*, volume 351 of *Leibniz International Proceedings in Informatics (LIPIcs)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2025. Article 109.
- [11] Pascal Massart. The tight constant in the Dvoretzky–Kiefer–Wolfowitz inequality. *The Annals of Probability*, 18(3):1269–1283, 1990.
- [12] Wim Vervaat. A relation between Brownian bridge and Brownian excursion. *The Annals of Probability*, 7(1):143–149, 1979.